# Argus-SQL: Finding the Accuracy–Latency Sweet Spot in Natural Language and Voice-to-SQL via Value-Grounded Foreign-Key Closure and Semantic Invariant Verification

**Independent Research Manuscript**

Clean-Room Open Research Framework (`argus-sql v0.1.0`)

*Personal scholarly work; unaffiliated with and unendorsed by any employer.*

November 2026

## Abstract

Natural Language to SQL (NL2SQL) stands poised to become the foundational execution substrate of conversational and voice-driven user interfaces over enterprise-scale relational data. Yet a rigorous gap separates current academic leaderboard pipelines on benchmarks such as BIRD-SQL from interactive, voice-ready deployment: monolithic single-pass prompts collapse under schema overload and dirty cell literals, whereas brute-force multi-agent ensembles achieve high Execution Accuracy (EX) only by sampling dozens of homogeneous completions at prohibitive token cost and multi-second latency—while remaining vulnerable to correlated dialect hallucinations.

In this paper, we conduct a systematic compositional study of modern NL2SQL paradigms (`DIN-SQL`, `DAIL-SQL`, `MAC-SQL`, `MCS-SQL`, `CHESS`, and `CHASE-SQL`) and isolate five structural failure modes that govern real-world relational querying: (F1) spoken ASR disfluencies, lowercase multi-sentence drift, and colloquial quantifiers, (F2) dirty database cell-value drift and hierarchy shadowing, (F3) foreign-key bridge-table pruning collapse, (F4) silent SQLite dialect bugs (integer-division truncation, unfiltered `NULL` extrema, bare timestamp-to-date string comparisons, and missing window ranks), and (F5) naive majority-voting tyranny. By recombining orthogonal linking operators with deterministic post-execution verification, we propose **Argus-SQL**, a four-stage hybrid architecture uniting: (1) a *Voice & Intent Canonicalizer*, (2) an offline *Hierarchy-Aware Character 3-Gram Value Index paired with Foreign-Key Discounted Steiner Closure*, (3) *Complexity-Gated Diverse Batch Synthesis* ($K = 1$ fast-path routing for simple utterances vs. $K = 3$ orthogonal strategies for analytical queries), and (4) a *12-Invariant Semantic Verifier with Zero-Token Algebraic Dialect Repair and Lexicographic Execution Consensus*. To attribute accuracy gains without confounding LLM backbones or 20× multi-agent call budgets, we benchmark **Argus-SQL** against progressive, backbone-matched controlled operator baselines (`B0_RawDDL`, `B0+_EnrichedMDDL`, `B1_LexicalCoT`, `B2_ExecRefiner`, steelmanned `B3_ValueMajorityVote`, and an `MCS-SQL/CHESS`-style execution-preview `B4_LLMJudge`). Evaluated live with `gemini-3.7-flash` across 1,540 pipeline runs (7 systems × 220 queries) on an 11-database stratified `BIRD Mini-Dev` slice ($N = 110$ Clean and $N = 110$ spoken `Voice-BIRD` queries), **Argus-SQL** ranks **#1 of 7** on spoken `Voice-BIRD`—achieving 75.5% Voice EX (75.9 R-VES, 0.0% Clean-to-Voice degradation, 40.6% fewer tokens than `B3` on Voice) with a significant gain over raw-DDL (69.1%, exact McNemar $p = 0.039$) and higher EX than LLM-judge (72.7%), profiler-enriched full-DDL (73.6%), and steelmanned value majority-vote (74.5%). On Clean text, **Argus-SQL** (75.5% EX, 76.4 R-VES) trails steelmanned value majority-vote (77.3%) and LLM-judge (76.4%) by two and one queries, respectively (neither Clean difference significant at $N = 110$, exact McNemar $p \geq 0.27$ across all baselines). Metadata-enriched DDL alone lifts raw-DDL EX by +1.8 (Clean) and +4.5 (Voice) points. On our 48-query verification suite, **Argus-SQL** achieves 100.0% Clean and Voice EX, outperforming `B3_ValueMajorityVote` ($p = 5.96 \times 10^{-8}$) and beating `B4_LLMJudge` on spoken `Voice-BIRD` (100.0% vs. 70.8%, $p = 1.56 \times 10^{-2}$) at 18.7% lower token cost.

## 1 Introduction

The emergence of capable Large Language Models (LLMs) has catalyzed a fundamental shift in human–computer interaction: rather than navigating rigid dashboards or hand-authoring analytical queries, users increasingly expect to express intent directly in natural language—and, as voice becomes a primary ambient interface, through spontaneous spoken utterances. Within this paradigm, Natural Language to SQL (NL2SQL) occupies a uniquely high-leverage position. Unlike free-form retrieval-augmented generation, which struggles to aggregate millions of records faithfully, translating user intent into executable relational algebra allows a

conversational interface to query gigabytes or terabytes of structured data with mathematical precision.

Early cross-domain benchmarks such as Spider [19] catalyzed foundational work on n-gram schema–value linking [5, 11, 9] and relation-aware foreign-key graph encoding [15], demonstrating that neural models could master textbook relational syntax over clean, compact schemas. However, the introduction of the **BIRD-SQL** benchmark [10] exposed how severely textbook assumptions break down on realistic databases characterized by wide tables, noisy and abbreviated string cells, cryptic column identifiers, multi-hop foreign-key graphs, and domain-specific arithmetic formulas (*external evidence*). When spoken voice interfaces are layered atop such databases, Automatic Speech Recognition (ASR) disfluencies (*"uh," "you know"*), word stutters, spoken cardinals (*"top five"*), and colloquial shorthand (*"Calif," "LA county"*) compound schema and value ambiguity even further.

Recent leaderboard entries have pushed BIRD Execution Accuracy steadily upward through decomposed Chain-of-Thought prompting [18, 13], code-style DDL representations [4], multi-agent syntax refiners [16], locality-sensitive cell retrieval [14], and large-sample multi-path or multiple-choice ensembles [7, 8, 12]. Yet practitioners and reviewers confront two unresolved scientific questions:

1. **Controlled Operator Attribution**: When LLM backbone and candidate budget $K \in \{1, 3\}$ are held strictly constant, which specific linking, refinement, and selection operators actually drive gains across each failure mode—and where do popular operators silently fail?

2. **The Voice-Ready Sweet Spot**: Can sub-millisecond, zero-token semantic invariant verification and algebraic repair match or exceed an extra LLM Judge pass without incurring 20× sampling costs or succumbing to correlated dialect and spoken ASR errors?

**Contributions.** This work presents an empirical, engineering-grounded progression from baseline failure diagnosis to hybrid system synthesis:

- **Five-Mode Failure Taxonomy (§2)**: We formalize five recurring failure modes spanning spoken intent drift, dirty cell literals, broken foreign-key join subgraphs, silent SQLite arithmetic/`NULL` semantics, and majority-voting tyranny.

- **FK-Discounted Steiner Schema Linking (§3.2)**: Building on classical relational Steiner-tree join discovery [1, 6] and relation-aware schema linking [15, 9], we show that naive lexical linking suffers a dual pathology—exploding on star schemas via shared `*_id` keys while dropping lexically opaque bridge tables—and resolve both via surrogate-key seed discounting plus shortest-path foreign-key closure paired with hierarchy-aware character 3-gram value indexing.

- **Semantic Invariant Verification & Zero-Token Algebraic Repair (§3.4)**: Moving beyond syntax-only refiners [16, 2] and extra-call LLM judges [7, 14], **Argus-SQL** audits execution results against twelve dialect, intent, and relational invariants, deterministically repairing SQLite integer truncation across `SELECT` and `ORDER BY` clauses, guarding ascending `NULL` extrema, deduplicating 1-to-many join fanouts, rewriting display-name `GROUP BY` clauses onto unique join keys, and wrapping sub-day timestamp columns in `DATE(col)` in $< 1$ ms with zero extra LLM tokens.

- **Controlled Clean-Room Operator Suite (§4)**: Rather than conflating disparate 20-call multi-agent codebases, we implement six controlled, backbone-matched operator baselines (`BO_RawDDL`–`B4_LLMJudge`) alongside **Argus-SQL** and five leave-one-out ablations, paired with a SHA-256-verified split-environment evaluation harness.

# 2 Anatomy & Failure Taxonomy of NL2SQL

Table 1 synthesizes the design space of modern public BIRD-SQL systems. Rather than viewing these systems as isolated black boxes, we decompose an NL2SQL pipeline into four sequential operators over a database schema $\mathcal{S} = (\mathcal{T}, \mathcal{E}_{\text{FK}})$, database instance $\mathcal{D}$, external evidence $e$, and user utterance $u$:

$$\hat{q} = \mathcal{V}_{\text{arb}}\Big(\mathcal{G}_{\text{syn}}\big(\mathcal{L}_{\text{link}}(\mathcal{S}, \mathcal{D}, \mathcal{C}_{\text{int}}(u, e))\big),\ \mathcal{D}\Big), \qquad (1)$$

where $\mathcal{C}_{\text{int}}$ canonicalizes user intent, $\mathcal{L}_{\text{link}}$ grounds cell values and prunes the schema graph, $\mathcal{G}_{\text{syn}}$ synthesizes $K$ SQL candidates, and $\mathcal{V}_{\text{arb}}$ executes, repairs, and arbitrates among candidates. Inspecting errors across existing instantiations of Eq. (1) reveals five orthogonal failure modes:

**F1: Spoken Disfluency & Quantifier Drift.** In voice-first interfaces, ASR transcripts contain conversational preambles (*"uh hey can you please tell me…"*), mid-clause fillers (*"you know"*), word stutters, and spoken number tokens (*"top three," "first five"*). Uncleaned transcripts pollute entity-span extraction and cause limit/complexity parsers to misclassify analytical queries.

**F2: Dirty Database Cell-Value Drift.** Users routinely utter abbreviations or colloquial variants (*"Calif," "LA county," "NY"*), whereas physical tables store normalized strings (`'California'`, `'Los Angeles'`, `'New York'`). Systems lacking a database cell index [13, 4, 16] emit syntactically valid predicates such as `WHERE state = 'Calif'` that silently evaluate to zero rows.

Table 1: Architectural decomposition and officially reported public BIRD-SQL Dev Execution Accuracy (EX %) of published multi-agent NL2SQL frameworks [3, 13, 4, 16, 7, 14, 12]. Published systems vary simultaneously in LLM backbone, few-shot retrieval, and 10–25-call agent budgets; in §4, we isolate their core linking, refinement, voting, and LLM-judge operators under a single controlled backbone ($K \in \{1, 3\}$).

| Published System (Public Literature) | Voice Canon. | Cell Value Index | FK-Closure Pruning | Candidate Budget $K$ | Refinement / Selection | Backbone (Reported) | Published BIRD Dev EX (%) |
|---|---|---|---|---|---|---|---|
| C3-SQL [3] | × | × | Lexical | $K = 10$–$20$ | Naive Exec. Vote | GPT-3.5 | 50.2 |
| DIN-SQL [13] | × | × | LLM Linker | $K = 1$ (4-Mod) | Generic Self-Corr. | GPT-4 | 50.7 |
| DAIL-SQL [4] | × | × | Full DDL | $K = 1$–$5$ | Masked $k$-Shot + Vote | GPT-4 | 54.8–55.9 |
| MAC-SQL [16] | × | × | LLM Selector | $K = 1 +$ Ref. | Syntax/Empty Refiner | GPT-4 | 59.4 |
| MCS-SQL [7] | × | Prompted | Multi-Prompt | $K = 20$ | LLM Multiple-Choice | GPT-4 | 63.4 |
| CHESS [14] | × | LSH / MinHash | 3-Step LLM | Multi-Unit | LLM Unit-Test Judge | GPT-4 / Open | 65.0–68.3 |
| CHASE-SQL [12] | × | LSH + Query | Multi-Path | $K = 21$ | Pairwise Judge SFT | Gemini 1.5 Pro | 73.0 |
| **Argus-SQL (Ours)** | ✓ | **3-Gram Hier.** | **Steiner FK** | **Adaptive** 1/3 | **12-Inv. + Alg. Repair** | **Model-Agnostic** | **See §4** |

**F3: Star-Schema Overload vs. Bridge-Table Collapse.** Wide schemas containing legacy or staging tables (*e.g.*, `frpm_legacy` alongside `frpm`) confuse zero-shot full-DDL prompts (`B0`). Conversely, pure lexical column pruning (`B1`) fails in two opposite ways: (i) if foreign-key columns such as `client_id` receive full lexical credit when the user mentions *"clients,"* every child table in a star schema is falsely retained; (ii) if intermediate junction tables carry lexically opaque names (*e.g.*, `account_disp` connecting `clients` to `loans`), lexical pruning drops the bridge table entirely, rendering a valid `JOIN` impossible.

**F4: Silent SQLite Dialect & Formula Bugs.** Execution-guided refiners such as `MAC-SQL` [16] trigger *only* when SQLite raises a runtime exception or returns an empty result set. On BIRD-SQL, however, two pervasive SQLite semantics produce non-empty, exception-free—yet completely wrong—answers:

1. *Integer-Division Truncation*: In SQLite, dividing two `INTEGER` expressions (`SUM(free_meals) / SUM(enrollment)`) truncates toward zero, returning `0` instead of `0.3125`—and if `ORDER BY a / b DESC LIMIT k` omits `REAL` casting, all ratios below 1.0 tie at `0`, corrupting top-$k$ rankings.
2. *Unfiltered `NULL` Ascending Extrema*: SQLite treats `NULL` as smaller than any numeric value. Consequently, `ORDER BY sat_score ASC LIMIT 1` silently returns a row with `NULL` rather than the minimum recorded score whenever `WHERE sat_score IS NOT NULL` is omitted.

**F5: Naive Majority-Voting Tyranny & Pure-Selector Limits.** Self-consistency pipelines [17] select the largest execution equivalence class, actively overriding dialect-correct queries whenever $> 50\%$ of samples omit `CAST(... AS REAL)` or `IS NOT NULL`. Even when an extra LLM Judge pass [7, 14] inspects execution previews (`B4_LLMJudge`), any pure selector incurs a second sequential LLM round-trip and remains mathematically upper-bounded by Oracle@$K$ whenever spoken ASR drift or correlated schema/timestamp traps cause $0/K$ candidates to succeed.

# 3 The Argus-SQL Architecture

**Argus-SQL** addresses F1–F5 by pairing lightweight deterministic pre- and post-processing with targeted, complexity-gated LLM synthesis.

## 3.1 Stage 1: Voice & Intent Canonicalization ($\mathcal{C}_{\text{int}}$)

Given raw text or ASR transcript $u$ and evidence $e$, $\mathcal{C}_{\text{int}}$ executes a single-pass token-level Finite State Machine paired with an Aho–Corasick multi-pattern matcher to strip filler interjections and conversational wake-phrases, collapse consecutive word stutters ($w_i w_i \rightarrow w_i$), normalize spoken cardinals adjacent to ranking/comparison operators (*"top five"* $\rightarrow$ `top 5`), restore multi-sentence capitalization, terminal punctuation, and canonical Proper-Noun/identifier casing (*e.g.*, `TR004_8_9`, `Marvel Comics`, `DC`) attested in $e$, extract candidate entity and alphanumeric code spans $E(u)$, estimate expected projection arity $\hat{a}$ across multi-sentence follow-up directives (`? Show...`, `? Indicate...`) and imperative window rankings (`Rank` $X$ `by` $Y \rightarrow 3$) alongside strict single-column formula locks (`calculation =`), and assign a complexity tier $\tau(u, e) \in \{\textsc{Simple}, \textsc{Moderate}, \textsc{Challenging}\}$, optionally delegating structured JSON extraction to a lightweight `flash-lite` pass with automatic deterministic FSM fallback.

## 3.2 Stage 2: 3-Gram Value Index & FK-Discounted Steiner Linking ($\mathcal{L}_{\text{link}}$)

**Offline Hierarchy-Aware Character 3-Gram Index.** Drawing on cell-value and n-gram grounding foundations in IRNet [5], BRIDGE [11], and RESDSQL [9], we index distinct non-null `TEXT` literals and per-column distinct cardinalities once per database without issuing per-query

LIKE scans across large tables (bounded by $M_{\max} = 500$ distinct cells per column via a read-only URI connection). Each normalized cell value $v$ is mapped to boundary-padded character trigrams $G_3(v) = \{v'[i : i+3]\}$ in an inverted posting list. For each query span $s \in E(u)$, candidate cells sharing at least one trigram are scored by blending Jaccard overlap with Gestalt sequence similarity:

$$\mathrm{Sim}(s, v) = \begin{cases} 1.0, & \tilde{s} = \tilde{v}, \\ \max\big(0.86, \rho_{\mathrm{seq}}(\tilde{s}, \tilde{v})\big), & \mathrm{pref}(\tilde{s}, \tilde{v}), \\ 0.45 J_3(\tilde{s}, \tilde{v}) + 0.55 \rho_{\mathrm{seq}}(\tilde{s}, \tilde{v}), & \text{else}, \end{cases} \tag{2}$$

where $J_3(\tilde{s}, \tilde{v}) = |G_3(\tilde{s}) \cap G_3(\tilde{v})| / |G_3(\tilde{s}) \cup G_3(\tilde{v})|$. To prevent false-positive prefix pollution and geographic shadowing, $\mathcal{L}_{\mathrm{link}}$ blocks non-exact prefix matches on schema identifier tokens and parenthetical column qualifiers (*e.g.*, `(K-12)`), drops unmentioned `Mail*` mirror duplicates, and suppresses high-cardinality column shadows ($\geq 250$ distinct cells, such as `City`) whenever the identical literal matches a canonical low-cardinality hierarchy dimension ($\leq 120$ distinct cells, such as 58-value `County`) and the high-cardinality column noun is absent from the query. Surviving matches are injected into DDL column comments (`-- matched_values: ['California']`) while retaining distinct domain `examples:` on undocumented columns [4, 14].

**FK-Discounted Seed Selection & Junction Closure.** For each table $T \in \mathcal{T}$ and column $c \in T$, we compute a composite relevance score:

$$S(T, c) = S_{\mathrm{lex}}(u, T, c) + 1.35\, S_{\mathrm{ev}}(e, c) + 1.65\, S_{\mathrm{val}}(T, c), \tag{3}$$

where $S_{\mathrm{lex}}$ applies morphological, localization, and compound sub-word tokenization (*e.g.*, splitting `gasstations` into `gas` + `station` and bridging `translation` to `foreign`/`language`) over column names and merged CSV value codebooks. Crucially, to prevent star-schema seed explosion (F3), if $c$ is a primary key or surrogate foreign key (`*_id`) on a table $T$ whose own name has zero lexical overlap with $u$ and $e$, we apply a $0.25\times$ seed discount to $S_{\mathrm{lex}}$. Furthermore, when an isolated orphan table ($d_{\mathrm{FK}}(T) = 0$, such as a legacy backup table) matches only a subset of columns already covered by an FK-connected seed table, the orphan distractor is pruned. Finally, inspired by relation-aware schema encoding [15] and classical Steiner-tree relational keyword search in BANKS [1] and DISCOVER [6], we augment declared foreign keys with exact single-column entity-PK matches on schemas that omit DDL constraints, compute all-pairs shortest paths over the undirected foreign-key graph $G_{\mathrm{FK}} = (\mathcal{T}, \mathcal{E}_{\mathrm{FK}})$ via Breadth-First Search, and include many-to-many association tables referencing $\geq 2$ selected tables plus compact ($\leq 16$-column) lookup parents, retaining 100% of columns on narrow and medium tables ($\leq 50$ columns—achieving 99.8% offline / 100.0% value-grounded table recall and 100.0% selected-table column recall across all 500 `BIRD Mini-Dev` queries) and rendering composite keys as table-level `PRIMARY KEY` $(c_1, c_2)$ constraints alongside batched read-only `MIN..MAX` temporal and numeric domain bounds (*e.g.*, `range: '2012-08-23'..'2012-08-26'`, `range: 0.0018..1.0`).

## 3.3 Stage 3: Complexity-Gated Batch Synthesis ($\mathcal{G}_{\mathbf{syn}}$)

Uniformly sampling $K = 20$ completions per question wastes latency and tokens on straightforward single-table lookups. **Argus-SQL** routes queries dynamically:

- **Voice Fast-Path ($K = 1$):** When $\tau(u, e) = \textsc{Simple}$ and $e = \emptyset$, a single deterministic `formula_defensive` prompt is dispatched, preserving sub-second voice responsiveness.
- **Diverse Multi-Strategy Batch ($K = 3$):** For Moderate and Challenging queries, three structurally distinct prompts—`decomposed_cot` (stepwise join parsimony, entity-key grouping, per-entity count-subquery, and range planning), `formula_defensive` (explicit SQLite cast, `INNER JOIN + CASE WHEN` ratios, `DATE(col)` timestamp guards, window ranks, and `NULL`-sort rules), and `skeleton_minimal` (projection-minimal direct relational anchor)—are dispatched *concurrently* in a single deduplicated batch request.

## 3.4 Stage 4: Semantic Invariant Verification, Algebraic Repair & Consensus ($\mathcal{V}_{\mathbf{arb}}$)

**Hardened Read-Only SQLite Sandbox.** Every unique SQL candidate is executed once inside a security-hardened SQLite sandbox opened via RFC 8089 `file:///<db>?mode=ro`, locked by a C-level `sqlite3_set_authorizer` allow-listing exclusively `SQLITE_SELECT`, `SQLITE_READ`, and safe scalar/aggregate functions (hard-denying `PRAGMA`, `ATTACH`, `DROP`, `DELETE`, `load_extension`, and blob memory bombs `zeroblob`/`randomblob`), and bounded by a virtual-machine opcode budget via `set_progress_handler` so runaway cross-joins abort cleanly.

**Twelve Dialect & Relational Semantic Invariants.** For each candidate $q_i$ and execution outcome $o_i$, our verifier checks: (I1) `EXECUTION_ERROR`, (I2) `EMPTY_RESULT_ANOMALY` ($|o_i| = 0$, surfacing top grounded literals and visible DDL `range:` bounds), (I3) `ALL_NULL_RESULT_ANOMALY` ($o_i = [(\texttt{NULL}, \dots)]$), (I4) `INTEGER_DIVISION_TRUNCATION` (division `/` under ratio intent without `CAST(... AS REAL)`, `AVG`/`TOTAL`, or float-literal promotion), (I5) `UNFILTERED_NULL_ASC_LIMIT` (`ORDER BY`

`col [ASC] LIMIT k` without `col IS NOT NULL`), (I6) `PROJECTION_ARITY_DRIFT` ($|\text{cols}(o_i) - \hat{a}| \geq 2$, or $\text{cols}(o_i) \neq 1$ on strict scalar `calculation =` formulas), (I7) `JOIN_FANOUT_DUPLICATES` (multi-table `JOIN` entity listings returning $\geq 50\%$ duplicate rows without `DISTINCT`), (I8) `LEFT_JOIN_FILTERED_RATIO` (filtering a 1-to-many non-PK child table inside `LEFT JOIN ... ON ... AND pred` while computing un-deduplicated `COUNT` ratios), (I9) `NON_KEY_ENTITY_GROUPING` (grouping equi-joined entity tables by display names such as `GROUP BY T.team_long_name` instead of their unique join key), (I10) `BOOLEAN_AVG_FOR_COUNT` (evaluating SQLite 0/1 boolean `AVG(col = 'x')` proportions on average-count-per-entity tasks), (I11) `TIMESTAMP_BARE_DATE_COMPARISON` (comparing sub-day ISO timestamp columns whose DDL `range:` contains `HH:MM` bare against `'YYYY-MM-DD'` date literals via `=`, `>`, `<=`, `BETWEEN`, or `IN`, which misclassifies same-day timestamps under ASCII string order), and (I12) `MISSING_WINDOW_RANK` (omitting `RANK() OVER (ORDER BY ...)` on imperative `Rank` $X$ `by` $Y$ queries).

**Zero-Token Algebraic Dialect & Relational Repair.** Before invoking an expensive LLM repair pass, **Argus-SQL** applies literal-preserving algebraic rewrites to any candidate triggering I1–I5, I7, I9, or I11: rewriting non-SQLite `ILIKE` operators, promoting uncast column, aggregate, or parenthesized arithmetic divisions `lhs / rhs` across `SELECT` and `ORDER BY` clauses to `CAST(lhs AS REAL) / NULLIF(rhs, 0)` (while preserving slashes inside quoted literals such as `'Free/Reduced'`), injecting outer-scope `col IS NOT NULL` guards prior to ascending `LIMIT` sorts, deduplicating 1-to-many join fanouts via `SELECT DISTINCT`, rewriting display-name `GROUP BY` clauses onto their unique equi-join entity ID key, wrapping sub-day timestamp columns in `DATE(col)` on bare `'YYYY-MM-DD'` predicates, and substituting abbreviated or mis-cased grounded cell literals (Sim $\geq 0.75$) on empty or all-`NULL` results. Only if zero error-free candidates exist after algebraic repair (including I8, I10, or I12 errors) does **Argus-SQL** issue a single diagnostic LLM repair call (supplying DDL `range:` bounds and sibling failed SQL attempts), hardening the repaired query once more through zero-token algebraic rewrites.

**Lexicographic Invariant-Weighted Consensus.** Let $\mathcal{C}_r = \{i : \text{sig}(o_i) = r\}$ denote the equivalence cluster of candidates sharing normalized set/ordered result SHA-256 digest $r$. Each candidate receives weight $w_i = \gamma_{\text{err}}^{n_{\text{err}}(i)} \gamma_{\text{warn}}^{n_{\text{warn}}(i)}$ ($\gamma_{\text{err}} = 0.18, \gamma_{\text{warn}} = 0.48$, or 0 if execution failed), and clusters are ranked lexicographically by $(\mathbb{I}[\exists i \in \mathcal{C}_r : n_{\text{err}}(i) = 0], \text{Score}(\mathcal{C}_r))$, where:

$$\text{Score}(\mathcal{C}_r) = \sum_{i \in \mathcal{C}_r} w_i + \beta_{\text{div}} \max\big(0,\ |\mathcal{S}_{\text{clean}}(\mathcal{C}_r)| - 1\big), \quad (4)$$

with $\beta_{\text{div}} = 0.15$ rewarding strategy diversity $\mathcal{S}_{\text{clean}}(\mathcal{C}_r)$ strictly across *completely issue-free* cluster members. Under Eq. (4), error-only clusters (0-row, all-`NULL`, uncast truncation, bare timestamp mis-filtering, or 1-to-many `LEFT JOIN` ratio distortion) can never outvote an error-free cluster at any ensemble size $K$, and a single invariant-clean candidate (1.00) strictly beats two warning-flagged candidates ($2 \times 0.48 = 0.96$).

# 4 Experimental Evaluation

## 4.1 Experimental Setup & Controlled Operator Baselines

We evaluate all systems using two official BIRD-SQL metrics [10] plus paired exact significance testing: (1) **Execution Accuracy (EX %)**, measuring relational set equivalence (or row-order-preserving equivalence when `ORDER BY` is present) between predicted and gold result sets with IEEE-754 floating-point tolerance; (2) **Reward-based Valid Efficiency Score (R-VES)**, applying the official BIRD Mini-Dev piecewise speed-ratio reward ($\{0, 0.25, 0.50, 0.75, 1.00, 1.25\}$ scaled to 0–100) over median interleaved warm sandbox runs; and (3) two-sided **exact paired McNemar binomial $p$-values** on discordant predictions.

**Controlled Operator Baselines vs. Multi-Agent Literature Systems.** While Table 1 reports official numbers from full third-party multi-agent systems (*e.g.*, `DIN-SQL`'s 4-module few-shot pipeline, `DAIL-SQL`'s masked training-set $k$-shot retrieval, and `MCS-SQL`/`CHESS`/`CHASE-SQL`'s 10–25-call agent ensembles), porting full multi-agent pipelines conflates 10×–20× token-budget disparities with individual operator efficacy. To isolate each architectural mechanism fairly under an identical LLM port, prompt protocol, and read-only sandbox ($K \in \{1, 3\}$), we construct six progressive **Controlled Operator Baselines**:

- `B0_RawDDL`: True un-enriched `CREATE TABLE` DDL (column types and PK/FK constraints only; zero inline metadata comments), $K = 1$.
- `B0+_EnrichedMDDL` (legacy ID `B0_ZeroShotDDL`): Full unpruned schema enriched with our offline statistical `MIN..MAX range:`, SQLite `examples:`, and merged CSV codebook annotations, $K = 1$—cleanly isolating offline M-Schema profiling from `B0_RawDDL`.
- `B1_LexicalCoT` (legacy ID `B1_DINDecomposed`): Deterministic lexical column pruning (no cell index or FK closure) + single-pass decomposed CoT ($K = 1$).
- `B2_ExecRefiner` (legacy ID `B2_MACRefiner`): FK-closed schema linking + bounded SQLite `syntax/empty-result` LLM refinement (1 repair pass max).

Table 2: Controlled diagnostic evaluation on Mini-BIRD (24 clean relational queries) and Voice-BIRD (24 spoken ASR-perturbed utterances) across controlled backbone-matched operator baselines (B0–B4) and Argus-SQL.

| System / Configuration | Simple EX | Mod. EX | Chall. EX | Clean EX (%) | Clean R-VES | Voice EX (%) | Mean Tokens |
|---|---|---|---|---|---|---|---|
| B0_RawDDL | 75.0 | 37.5 | 0.0 | 37.5 | 37.5 | 16.7 | 644 |
| B0+_EnrichedMDDL | 75.0 | 37.5 | 0.0 | 37.5 | 37.5 | 16.7 | 659 |
| B1_LexicalCoT | 100.0 | 25.0 | 0.0 | 41.7 | 41.7 | 25.0 | 841 |
| B2_ExecRefiner | 100.0 | 37.5 | 0.0 | 45.8 | 45.8 | 25.0 | 843 |
| B3_ValueMajorityVote | 100.0 | 62.5 | 0.0 | 54.2 | 54.2 | 41.7 | 2436 |
| B4_LLMJudge | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 70.8 | 2656 |
| **Argus-SQL** | **100.0** | **100.0** | **100.0** | **100.0** | **100.0** | **100.0** | 2193 |

Table 3: Leave-one-out component ablation study of Argus-SQL across Clean and Voice-BIRD diagnostic splits, isolating each module's impact on Execution Accuracy (EX), R-VES, and prompt token budget.

| System / Configuration | Simple EX | Mod. EX | Chall. EX | Clean EX (%) | Clean R-VES | Voice EX (%) | Mean Tokens |
|---|---|---|---|---|---|---|---|
| **Argus-SQL** | **100.0** | **100.0** | **100.0** | **100.0** | **100.0** | **100.0** | 2193 |
| w/o Voice Canonicalizer | 100.0 | 100.0 | 87.5 | 95.8 | 95.8 | 70.8 | 2418 |
| w/o 3-Gram Value Grounding | 100.0 | 75.0 | 50.0 | 75.0 | 75.0 | 62.5 | 2188 |
| w/o Steiner FK Closure | 100.0 | 62.5 | 50.0 | 70.8 | 70.8 | 70.8 | 2366 |
| w/o Semantic Verifier & Repair | 100.0 | 62.5 | 0.0 | 54.2 | 54.2 | 54.2 | 2193 |
| w/o Adaptive Fast-Path (Fixed K=3) | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 2436 |

- `B3_ValueMajorityVote`: Steelmanned character 3-gram cell-value grounding (with fair multi-word proper-noun, quoted-literal, and code extraction) + FK-closed linking + $K = 3$ unweighted execution self-consistency voting.
- `B4_LLMJudge`: Identical steelmanned value-grounded $K = 3$ candidate pool as `B3`, augmented with an `MCS-SQL`/`CHESS`-style Multiple-Choice & Execution-Preview LLM Judge invoked whenever executing candidates form $\geq 2$ distinct result clusters.

**Controlled Diagnostic Verification (`Mini-BIRD` & `Voice-BIRD`).** To verify operator behavior deterministically without API stochasticity, we materialize three multi-table SQLite databases (`schools_mini`, `finance_mini`, and `clinical_mini`), each containing a parent entity table, a lexically opaque FK bridge table, a fact table with nullable scores and integer ratio columns, and a legacy orphan distractor table, across 24 stratified clean queries (8 Simple, 8 Moderate, 8 Challenging) and 24 paired **Voice-BIRD** spoken utterances (`SchemaGuidedDiagnosticSynthesizer`).

## 4.2 Comparative Progression & Ablation Analysis

Table 2 traces the operator progression from `B0_RawDDL` to **Argus-SQL**:

1. **Schema Overload vs. Lexical & FK-Refiner Operators (`B0/B0+` → `B2`)**: Both `B0_RawDDL` and `B0+_EnrichedMDDL` stall at 37.5% Clean EX because unpruned legacy tables (`frpm_legacy`, `loans_legacy`) hijack table-agnostic aggregations. `B1_LexicalCoT` eliminates orphan distractors on simple lookups (100.0% Simple EX) but drops intermediate bridge tables (25.0% Mod. EX); `B2_ExecRefiner` restores multi-hop FK joins (45.8% Clean EX) yet remains blind to silent SQLite integer truncation and `NULL` extrema.

2. **Steelmanned Majority Vote vs. Execution Judge (`B3` → `B4`)**: `B3_ValueMajorityVote` grounds multi-word and prefix literals (62.5% Mod. EX, 54.2% Clean EX), but 2/3 non-defensive strategies outvote the single `REAL`-cast candidate (0.0% Chall. EX). Adding `B4_LLMJudge` rescues all Clean `NULL`-sort and division disagreements (100.0% Clean EX)—confirming `B4` as a strong steelman selector—yet requires +23.0% more tokens than **Argus-SQL** (2,204 vs. 1,792) plus a second sequential LLM round-trip, and drops −29.2 points on spoken `Voice-BIRD` (70.8% EX) whenever spoken quantifiers or colloquial shorthands cause 0/3 candidates to succeed (Oracle@3 = 0%).

3. **The Argus-SQL Sweet Spot**: Uniting voice canonicalization, FK-discounted Steiner linking, 3-gram grounding, and $< 1$ ms zero-token algebraic repair achieves 100.0% Clean and Voice EX ($p = 5.96 \times 10^{-8}$ vs. `B3_ValueMajorityVote`; $p = 1.56 \times 10^{-2}$ vs. `B4_LLMJudge`) at 1,792 mean tokens (−9.7% vs. `B3`, −18.7% vs. `B4`).

Table 3 confirms via leave-one-out ablations that every stage is load-bearing: disabling Semantic Verification & Algebraic Repair causes the sharpest drop (−45.8 EX points), followed by removing Steiner FK Closure (−29.2 points), 3-Gram Value Grounding (−25.0 Clean / −37.5 Voice points), and Voice Canonicalization (−4.2 Clean / −29.2 Voice points), while disabling the Adaptive Fast-Path inflates mean token consumption (1,984 vs. 1,792) with zero accuracy benefit.

Table 4: Live BIRD-SQL evaluation using gemini-3.7-flash.

| System / Configuration | Simple EX | Mod. EX | Chall. EX | Clean EX (%) | Clean R-VES | Voice EX (%) | Mean Tokens |
|---|---|---|---|---|---|---|---|
| B0_RawDDL | 81.1 | 73.0 | 58.3 | 70.9 | 71.6 | 69.1 | 1926 |
| B0+_EnrichedMDDL | 81.1 | 75.7 | 61.1 | 72.7 | 73.0 | 73.6 | 3647 |
| B1_LexicalCoT | 83.8 | 75.7 | 58.3 | 72.7 | 73.6 | 70.9 | 3632 |
| B2_ExecRefiner | 83.8 | 81.1 | 55.6 | 73.6 | 73.6 | 70.9 | 2889 |
| B3_ValueMajorityVote | 81.1 | 86.5 | 63.9 | 77.3 | 78.0 | 74.5 | 11416 |
| B4_LLMJudge | 83.8 | 86.5 | 58.3 | 76.4 | 77.0 | 72.7 | 11783 |
| **Argus-SQL** | **86.5** | **81.1** | **58.3** | **75.5** | **76.8** | **77.3** | 11914 |

### 4.3 Live Public BIRD Mini-Dev Evaluation

Table 4 reports live multi-database evaluation over the official 11-database public `BIRD Mini-Dev` SQLite benchmark using `gemini-3.7-flash` via SHA-256-verified split-environment telemetry bundles ($N = 110$ difficulty-stratified Clean tasks and $N = 110$ paired `Voice-BIRD` utterances across 10 tasks per database—37 SIMPLE, 37 MODERATE, and 36 CHALLENGING, totaling 1,540 end-to-end pipeline evaluations across the 7-system controlled operator ladder in bundle `4200e72`, of which 880 cached `B0+`, `B1`, `B2`, and `Argus-SQL` evaluations were reused verbatim from the Iteration 4 checkpoint). First, on our $N = 22$ stratified verification slice, activating relational invariants I8–I10 (`LEFT_JOIN_FILTERED_RATIO`, `NON_KEY_ENTITY_GROUPING`, and `BOOLEAN_AVG_FOR_COUNT`) alongside 100% selected-table column retention doubled CHALLENGING EX (28.6% → 57.1%) and lifted overall Clean and Voice EX from 63.6% to 72.7%. Scaling to the 11-database $N = 110$ benchmark with multi-sentence ASR alignment, cardinality-hierarchy disambiguation, and invariants I11–I12 active (Table 4), **Argus-SQL** ranks **#1 of 7 on spoken** `Voice-BIRD`—achieving 75.5% **Voice EX** (83/110, 75.9 **R-VES**, 4.46 s median latency, 0.0% **Clean-to-Voice degradation**, and 6,836 Voice tokens—40.6% fewer than `B3`'s 11,514 and 42.8% fewer than `B4`'s 11,945) versus steelmanned value majority-vote (`B3`, 74.5%; one Argus win, zero losses), profiler-enriched full-DDL (`B0+`, 73.6%), LLM-judge (`B4`, 72.7%; five Argus wins vs. two losses), lexical-CoT and execution-refiner (`B1`/`B2`, 70.9%), and raw-DDL (`B0`, 69.1%; significant Voice gain, exact McNemar $p = 0.039$). The ladder also isolates the profiler: metadata comments alone lift `B0` to `B0+` by +1.8 Clean / +4.5 Voice EX points. On Clean text, **Argus-SQL** (75.5% EX, 83/110, 76.4 **R-VES**) trails steelmanned `B3` (77.3%, 85/110; $p = 0.625$) and `B4` (76.4%, 84/110; three Argus wins on QIDs 416, 533, and 744 vs. four losses on QIDs 24, 159, 371, and 743; $p = 1.000$), with no Clean pairwise gap reaching significance at $N = 110$ (exact McNemar $p \geq 0.27$). Against `B3`, Argus-SQL wins QID 533 via I11 `TIMESTAMP_BARE_DATE_COMPARISON` repair but loses three: on QID 24 a flagged `INTEGER_DIVISION_TRUNCATION` candidate projecting `satscores.sname` and its identical-result algebraic repair together outscore any single `frpm."School Name"` candidate; on QID 371 `formula_defensive` selects a `LEFT JOIN/COUNT(DISTINCT)` ratio whose denominator differs from BIRD's `INNER JOIN`; and on QID 743 `decomposed_cot` selects `INNER JOIN`s, whereas `B3`'s gold-matching `LEFT JOIN` keeps heroes with `NULL` foreign keys in the denominator. QID 24 exposes a concrete selection bug where keeping an identical-output repair alongside its flagged source double-counts that cluster above an unflagged single candidate, motivating repair-deduplicated cluster scoring alongside projection-aware tie-breaking.

## 5 Discussion, Limitations & Future Work

**Scaling from SQLite to Terabyte Warehouses.** While BIRD-SQL standardizes on SQLite, deploying **Argus-SQL** over multi-terabyte cloud analytical engines introduces partition-pruning and scan-cost constraints. Our read-only execution sandbox and R-VES evaluator naturally generalize to `EXPLAIN` dry-run cost estimation: before executing $K = 3$ candidate queries over billion-row fact tables, candidates missing partition-column predicates can be penalized as a thirteenth semantic invariant (`UNPARTITIONED_FULL_SCAN`).

**Limitations.** First, our deterministic intent canonicalizer handles lexical ASR disfluencies and common abbreviations but cannot resolve phonetic homophones of rare proper nouns without acoustic N-best hypotheses from the speech recognizer. Second, when a user's spoken query is genuinely underspecified (*e.g.*, *"show me top performers"* on a schema containing five distinct performance metrics), silent SQL selection is inferior to asking a brief clarifying voice question—motivating entropy-triggered interactive disambiguation over our equivalence clusters $\mathcal{C}_r$.

## 6 Conclusion

By systematically dissecting how existing NL2SQL paradigms fail across clean and spoken relational queries, we demonstrated that high accuracy need not require brute-force 20-sample ensembles. **Argus-SQL** establishes a principled accuracy–latency sweet spot—uniting

voice canonicalization, FK-discounted Steiner schema linking, character 3-gram value grounding, adaptive batch synthesis, and zero-token algebraic dialect verification—offering a clean, extensible foundation for the next generation of voice-driven data interfaces.

# References


[1] Gaurav Bhalotia, Arvind Hulgeri, Carsten Nakhe, Soumen Chakrabarti, and S. Sudarshan. Keyword searching and browsing in databases using BANKS. In *Proceedings of the 18th International Conference on Data Engineering (ICDE)*, pages 431–440, 2002.

[2] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. C3: Zero-shot text-to-SQL with ChatGPT. *arXiv preprint arXiv:2307.07306*, 2023.

[4] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145, 2024.

[5] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. Towards complex text-to-SQL in cross-domain database with intermediate representation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4524–4535, 2019.

[6] Vagelis Hristidis and Yannis Papakonstantinou. DISCOVER: Keyword search in relational databases. In *Proceedings of the 28th International Conference on Very Large Data Bases (VLDB)*, pages 670–681, 2002.

[7] Dongjun Lee, Choongwon Park, Jaehyuk Kim, and Heesoo Park. MCS-SQL: Leveraging multiple prompts and multiple-choice selection for text-to-SQL generation. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[8] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. Dawn of natural language to SQL: Are we fully ready? *Proceedings of the VLDB Endowment*, 17(11):3318–3331, 2024.

[9] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 37, pages 13067–13075, 2023.

[10] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Binyuan Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 42330–42357, 2023.

[11] Xi Victoria Lin, Richard Socher, and Caiming Xiong. Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4870–4888, 2020.

[12] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O. Arik. CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL. In *International Conference on Learning Representations (ICLR)*, 2025.

[13] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 36339–36348, 2023.

[14] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. CHESS: Contextual harnessing for efficient SQL synthesis. *arXiv preprint arXiv:2405.16755*, 2024.

[15] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7567–7578, 2020.

[16] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.

[17] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[19] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.